

Security discussions in quantum software projects on GitHub[☆]Gastón Márquez^{a, ID, *}, Muhammad Waseem^b, Tommi Mikkonen^c^a Departamento de Ciencias de la Computación y Tecnologías de la Información, Universidad del Bío-Bío, Andrés Bello 720, Chillan, 3780000, Ñuble, Chile^b Faculty of Information Technology and Communication, Tampere University, Tampere, 33100, Finland^c Faculty of Information Technology, University of Jyväskylä, Jyväskylä, 40100, Finland

ARTICLE INFO

MSC:

0000

1111

Keywords:

Security

Quantum software engineering

Topic analysis

Repository mining

ABSTRACT

Quantum software engineering is an emerging field that leverages quantum computing and software development to address contemporary computational challenges. A significant concern in this domain is security, which has become a critical issue in quantum software engineering. Despite advances in quantum computing security, there is a lack of empirical evidence examining the primary security concerns of developers within the context of quantum software engineering. This paper presents a study that identifies, describes, and analyzes topics discussed by developers regarding open and closed issues of quantum software projects hosted on GitHub. Of the 18 identified projects, 2,264 filtered open and closed issues were obtained, of which 294 (13%) were related to security. Using the Latent Dirichlet Allocation algorithm, 15 topics were identified. Furthermore, we identified the key security concerns that developers addressed in the issues, the majority of which were oriented towards code failure, noise and modular validation. This study serves as a precedent for a practical analysis of the identification and characterization of security topics, as well as the initial insights into security design decisions that developers discuss in quantum software projects.

1. Introduction

Quantum software engineering is a rapidly developing field that integrates the principles of quantum computing with software development and methodology (Piattini et al., 2021). This discipline leverages the unique properties of qubits, or quantum bits, such as superposition and entanglement, to perform computations more efficiently than classical computers (Zhao, 2020). The primary objective of quantum software engineering is to design, develop, and optimize algorithms and programs that exploit these quantum capabilities to solve complex problems. As quantum computers become increasingly accessible and advanced, this field is expected to make significant contributions to areas such as cryptography, optimization, simulation of quantum systems, and machine learning (Ali et al., 2022).

Quantum software engineering embraces the practices, methodologies, and challenges of traditional software engineering. One of these challenges is the development of secure software (Fujdiak et al., 2019). The International Organization for Standardization (ISO) 25010 defines security as a quality attribute that reflects the extent to which a product or system safeguards information and data, allowing access to it by people or other products or systems in accordance with their types and levels of authorization (Iso, 2011). The development of secure software involves the implementation of processes that aim to ensure

that the software is resilient to security threats and vulnerabilities. This may include measures such as security requirements, security design decisions, encryption, and secure deployment, among others.

Similar to traditional software engineering, security has also become a concern in quantum software engineering (Arias et al., 2023). Security in quantum-software development is a critical consideration given the novel nature of quantum computing. Quantum software security issues differ from those of classical software due to the unique properties of quantum mechanics, which fundamentally alter the manner in which computations are performed, data is processed, and vulnerabilities are introduced or mitigated. Quantum software utilizes principles such as superposition, entanglement, and measurement, which transform the security landscape in ways that are not observed in classical software. For instance, quantum software operates with qubits, which are highly susceptible to errors due to even minor perturbations in their environment (referred to as “quantum noise” Clerk et al., 2010). Unlike classical bits, qubits can exist in a superposition of states, rendering them vulnerable to data corruption resulting from decoherence (loss of quantum state) and external interference.

The Talavera Manifesto (Piattini et al., 2020) emphasizes security and privacy by design, advocating for quantum software engineers to incorporate security practices directly into the development lifecycle.

[☆] Editor: Professor Yan Cai.

* Corresponding author.

E-mail address: gmarquez@ubiobio.cl (G. Márquez).

This approach encompasses the implementation of post-quantum cryptography for hybrid classical quantum systems, the development of secure quantum communication protocols, and the mitigation of risks associated with quantum hardware vulnerabilities (e.g., side-channel attacks). In this regard, there are several studies that address security, mainly with a focus on cryptography (Gisin et al., 2002). Additionally, some studies, such as Arias et al. (2023) and Alyami et al. (2021), have started to discuss security concerns from a quantum software engineering point of view. However, there is little evidence from studies that analyze security concerns from a pragmatic point of view. Moreover, there is limited information on research that looks at security concerns that developers discuss in quantum software development projects.

In this paper, we describe a study that identifies topics detected in open and closed issues in quantum software projects hosted on GitHub, as well as potential security concepts mentioned in these projects. In addition, we analyzed the nature of the questions asked by developers regarding the security aspects of quantum software development. We defined a research design that allowed us to identify 18 projects on GitHub that met our research objective. From this set of projects, we obtained 294 security-related issues out of a total of 2264 filtered issues, considering both open and closed issues. From this final set, we identified a set of 15 topics using the Latent Dirichlet Allocation algorithm (Blei et al., 2003) and studied the origin and nature of the questions associated with these issues as well as security concerns described by these issues.

This study's primary contribution lies in its exploration of topics related to security-related discussions among quantum software developers. By examining the issues raised, we obtained valuable insights into potential security problems and the main concerns associated with quantum programming languages and technologies. Additionally, the analysis discusses strategies employed by developers to address these security concerns in quantum software projects.

The remainder of this paper is organized as follows. Section 2 describes related work; Section 3 details the research design; Section 4 shows the results; Section 5 describes the discussion; Section 6 discusses the software engineering perspective; Section 7 details the threats to validity; and Section 8 concludes our study and discusses future work. Additionally, we created a repository¹ to replicate our study. In the repository, we describe the codes, the issues referenced in Section 4, files and dataset to replicate the methodology used in this research.

2. Related work

Alyami et al. (2021) employed a hybrid approach that combined the symmetric fuzzy analytic hierarchy process (FAHP) and fuzzy ideal solution similarity preference order technique (FTOPSIS) to explore software safety in quantum computing. Their findings offer valuable insights for future research in organizing software security and durability in the presence of quantum computers. Furthermore, the authors emphasized the need for extensive research in the context of current quantum cybersecurity. They identified the primary challenges of the quantum era as cryptographic methods that meet the demands of usability, flexibility, and security while maintaining user trust.

Aguado et al. (2017) suggest the incorporation of Quantum Key Distribution (QKD) systems with established protocols and methodologies to secure the network control plane in a software-defined networking (SDN) and network functions virtualization (NFV) environment. Furthermore, the authors describe an experimental investigation that explores a process in which QKD keys collaborate with classically generated keys to encrypt communications between the cloud and SDN platforms for configuring a secure shell service. In a similar vein, Nosouhi et al. (2023) proposed a framework that facilitates SDN-based networks to utilize QKD technology and provide a QKD security

service (i.e., a random key generation service) to network applications and security protocols in a practical and efficient fashion. In this proposed approach, secret keys are generated based on the distribution of quantum entanglement among QKD nodes deployed in the network.

Arias et al. (2023) have emerged as leaders in proposing recommendations for safeguarding quantum software development. The authors have underscored the dearth of resources, methodologies, and expertise within the quantum software community to guarantee secure development.

The studies addressed in this section discuss security issues related to software development from various perspectives. This study addresses security topics in quantum software projects through an analysis of developer discussions. These discussions frequently elucidate developers' perceptions, attitudes, and levels of awareness regarding security practices in the context of quantum software, providing insights into security trade-offs and risk mitigation strategies. Furthermore, the review of open and closed issues facilitates the analysis of the evolution of security concerns. This longitudinal perspective can aid in predicting future security requirements and inform the proactive development of quantum software security practices and tools. Moreover, the contribution of this research lies in delineating the perception of security from the perspective of quantum software developers to ascertain which security concerns tend to be most salient.

3. Research design

In this section, we provide an overview of the methodological approach employed in our study (see Fig. 1).

The purpose of our research is to investigate and analyze both open and closed issues pertaining to quantum software projects on Github and identify and evaluate security-related concerns and design decisions within the context of quantum software engineering.

3.1. Research questions

The research questions are described as follow:

- RQ1: *Which topics are detected in the issues?* The objective of this research question is to identify the number of topics that represent the issues. For each topic, this research question addressed the words associated with the topics to classify the topics.
- RQ2: *Which security concerns are discussed in each topic?* This research question intends to delve deeper into the security concerns addressed in the issues for each topic. For this purpose, we analyzed quantum-related vulnerabilities reported in the common vulnerability and exposure (CVE) to identify similarities with the issues of each topic
- RQ3: *What kinds of questions do developers ask about issues related to security topics?* Given the complexity of addressing security in the context of quantum, this research question aims to analyze the discussions made by developers in quantum projects. Therefore, this research question analyzes the questions asked in each issue related to security topics and classifies them according to their nature: "what?", "how?", "why?", and other.

3.2. Project selection

We formulated inclusion and exclusion criteria to identify quantum software projects on GitHub. The inclusion criteria we have defined are as follows:

- The project must explicitly describe the quantum-related functionality it implements. The purpose of this criterion is to know what scope the project addresses.
- The project must have active activity. For this, we define a range of activity within the last 18 months. This activity considers conversations on issues and pull requests.

¹ <https://doi.org/10.5281/zenodo.15660259>.

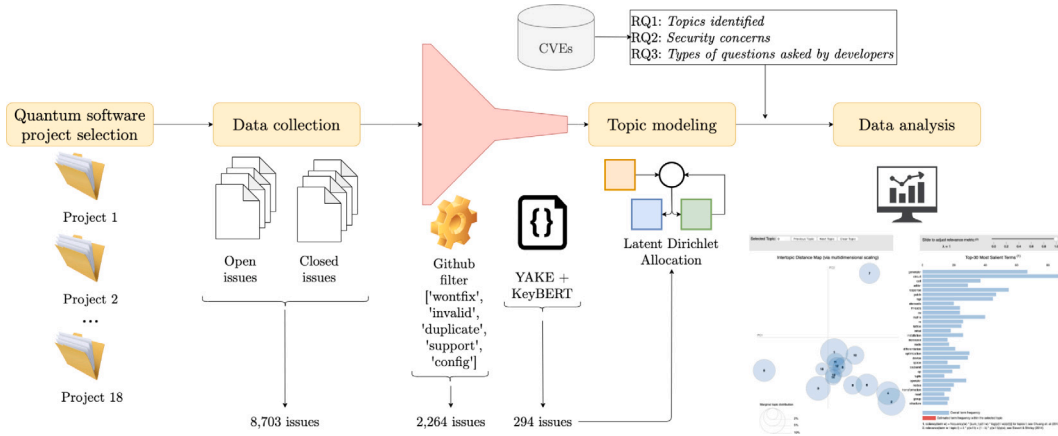


Fig. 1. Research process.

Table 1
Quantum projects details.

Project title	URL (https://github.com/)	Pull requests	Forks	Commits	Stars	Open	Closed
ProjectQ	/ProjectQ-Framework/ProjectQ	8	242	246	745	21	122
cp2k	/cp2k/cp2k	4	228	20,024	420	101	512
QUICK	/merzlab/QUICK	8	29	2518	47	23	94
nuSQuIDS	/arguelles/nuSQuIDS	3	28	849	15	7	11
DCore	/issp-center-dev/DCore	1	10	943	36	8	9
pennylane-cirq	/PennyLaneAI/pennylane-cirq	1	12	274	33	2	14
liboqs-java	/open-quantum-safe/liboqs-java	0	9	97	13	2	9
amazon-qldb-driver-go	/awslabs/amazon-qldb-driver-go	2	9	148	21	1	16
qc-atlas	/UST-QuAntiL/qc-atlas	3	0	216	3	12	19
aiida-gaussian	/nanotech-empa/aiida-gaussian	0	1	103	3	2	9
lattice-symmetries	/twesterhout/lattice-symmetries	0	2	280	12	1	1
braket	/naoki-yoshioka/braket	0	0	284	2	21	66
ModelHamiltonian	/theochem/ModelHamiltonian	0	1	30	2	5	9
qiskit	/Qiskit/qiskit	205	2417	9145	5410	818	4035
Cirq	/quantumlib/Cirq	51	1050	3962	4300	271	1980
pennylane	/PennyLaneAI/pennylane	92	615	4933	2400	227	1130
strawberryfields	/XanaduAI/strawberryfields	17	193	1171	768	23	116
OpenFermion	/quantumlib/OpenFermion	5	382	573	1549	76	205

- Available documentation. This criterion is important to understand the project and compile the code if necessary.
- Compilable backend. Projects must describe support for compiling infrastructure (such as Cirq and Braket) and must support quantum-oriented programming languages (such as Qiskit, Q#, among others).

The exclusion criteria are as follows:

- The project has no recent activity during the last 18 months.
- Poor documentation.
- Deprecated projects.
- Toy or prototype projects.
- Projects with basic or limited coding.

With respect to issues, we excluded those that (i) ask general questions, (ii) have not been answered for a long time, (iii) do not have sufficient information on the origin of the issue or cause, (iv) do not have any description, (v) have been closed without grounds, and (vi) are duplicated. Following the application of these criteria, we obtained a total of 18 quantum software projects (see Table 1). These projects described a diverse representation of the contemporary quantum software development ecosystem. They encompass widely adopted frameworks such as Qiskit, Cirq, and PennyLane, which foster activity and community engagement, underscoring their pivotal role in the research and application of quantum algorithms. Similarly, specialized projects, such as OpenFermion, ProjectQ, and cp2k, exemplify the integration between quantum simulation and other disciplines.

3.3. Data collection

In order to gather both open and closed issues, we generated a dataset that summarized each project's posts. This was achieved through the use of web scraping to automate the collection of issues. By applying this technique to the 18 selected projects, we obtained a total of 8703 issues.

Open and closed structures exhibit distinct characteristics. Some of these characteristics may pertain to configuration inquiries, configuration comments, duplicate questions, infrastructure support, among others. Consequently, we employ a filtering process on the issues retrieved via the GitHub API, which categorizes the issues. This process allows us to exclude issues with tags such as 'wontfix', 'invalid', 'duplicate', 'support', and 'config.' Following the application of this tag-based filter, a total of 2264 issues were identified.

To address security concerns within the dataset of 2264 issues, we refined the dataset by identifying security-related issues. This refinement was achieved through a two-phase approach utilizing the YAKE and KeyBERT algorithms. YAKE is an unsupervised algorithm that extracts keywords from documents using statistical methods (Campos et al., 2020). KeyBERT, on the other hand, is based on transformer models and employs the SentenceTransformer model to convert sentences or phrases into semantic vectors (embeddings), thereby extracting the most semantically representative phrases from a document (Grootendorst, 2020). By using them together, a balance between efficiency and depth is achieved: YAKE filters a large number of relevant candidates, and KeyBERT refines this selection, ensuring semantic relevance.

In the initial phase, we established a list of domain-specific security keywords, which functioned both as a direct match filter and as a semantic reference for similarity scoring. These keywords were derived from databases concerning vulnerabilities (CVE²), weaknesses (CWE³), and attack patterns (CAPEC⁴), in addition to studies on security in quantum computing (Saki et al., 2021; Ravi et al., 2022) and security tactics datasets (Santos et al., 2017).

Words: sha, security, breach, cybersecurity, cyber-security, cyber security, infosec, vulnerability, weakness, penetration, privacy, secure, protection, encryption, threat, incident, authentication, authorization, firewall, audit, patch, malware, risk, awareness, itsecurity, confidentiality, availability, integrity, attack, authentication, intrusion, fuzzing, compliance, cve, cwe, capec, trust, hashing, ssl, pinning, cryptography, homomorphic, quantum-safe, enclave, side-channel, overflow, injection, cross-site, lattice, polynomial, isogeny, kem, sike, nike, kyber, dilithium, signature, post-quantum, sphincs, md5, chap, pap, ldap, kerberos, eap, aes, blowfish, rsa, privilege, revoke, sftp, tls, vpn, ssh, block, isolate, restore, buffer, sandbox, cca, generator, timing, decryption, elliptic, supersingular, rlwe

We adjusted YAKE's parameters to extract up to 200 keywords, implementing a trigram limit, a deduplication threshold of 0.8, and a narrow context window to emphasize local contextual relevance, which is particularly advantageous for precise technical terms. In the second phase of our study, we employed a pre-trained model known as SentenceTransformer (all-MiniLM-L6-v2) to efficiently capture semantic embeddings (Issa et al., 2023). To focus on keyword ranking based solely on relevance, we deactivated the maximal marginal relevance and set the diversity parameter to zero.

In order to optimize both phases, we used regular expressions to match keywords against the list generated in the initial phase. In instances where a direct match was not identified, we calculated the cosine similarity between the embedding of the candidate keyword and the centroid of all security-related terms. Keywords that surpassed a similarity threshold of 0.7 were retained. This threshold was meticulously calibrated to achieve a balance between recall and precision, ensuring it was sufficiently high to eliminate noise while remaining low enough to capture semantically related variants (Park et al., 2020). Once the two phases have been applied, we obtain a dataset of 294 security-related issues. Algorithm 1 summarizes the steps performed to filter issues.

3.4. Topic modeling

To determine the topics, we utilized the Latent Dirichlet Allocation (LDA) algorithm (Blei et al., 2003), which accounts for stop words by excluding words that introduce noise during topic extraction. The algorithm posits that each document comprises a mixture of several topics, with each topic representing a probability distribution over words. LDA initially assigns random topics to the words in the documents and subsequently iteratively adjusts these assignments to maximize the likelihood of the observed data. The resultant output of the algorithm is a representation wherein each document possesses a probability distribution over topics, and each topic is characterized by a set of words with high associated probabilities.

Topic coherence assesses the semantic similarity of the most probable words for each topic (Röder et al., 2015). Higher coherence scores are generally associated with more interpretable topics. Conversely,

Algorithm 1 Security-Relevant Keyword Extraction using YAKE and KeyBERT

```

1: Input: Directory path  $D$ , Output file  $F$ , Security keyword list  $S$ 
2: Output: Security-related keywords per file in  $F$ 
3: Count total files in directory  $D$ 
4: Initialize keyword extractors:
5:   YAKE with  $n = 3$ ,  $dedupLim = 0.8$ ,  $top = 200$ ,  $windowSize = 1$ 
6:   KeyBERT with SentenceTransformer "all-MiniLM-L6-v2"
7: Clear content of output file  $F$ 
8: for each file  $f$  in directory  $D$  do
9:   Read text content  $T$  from file  $f$ 
10:  Extract keyword candidates  $C$  from  $T$  using YAKE
11:  Convert all  $C$  to lowercase
12:  Apply KeyBERT to re-rank  $C$ , using:
13:     $use\_mmr = \text{False}$ ,  $diversity = 0.0$ 
14:     $nr\_candidates = 150$ ,  $top_n = 50$ 
15:  Define regex pattern  $P$  from  $S$ 
16:  Compute centroid vector  $V_c$  from  $S$  using sentence embeddings
17:  Initialize filtered keyword list  $K = []$ 
18:  for each keyword  $(kw, sc)$  in KeyBERT results do
19:    if  $kw$  matches regex pattern  $P$  then
20:      Append  $(kw, sc)$  to  $K$ 
21:    else
22:      Compute cosine similarity  $sim$  between  $kw$  and  $V_c$ 
23:      if  $sim > 0.7$  then
24:        Append  $(kw, sc)$  to  $K$ 
25:      end if
26:    end if
27:  end for
28:  if  $K$  is not empty then
29:    Append file name  $f$  and keywords  $K$  to output file  $F$ 
30:  end if
31: end for

```

Python's Gensim library⁵ permits both alpha and eta to be set to "auto", enabling the model to automatically adjust these parameters during training based on the data, rather than utilizing predefined fixed values. In this configuration, the model dynamically adjusts the alpha values during training using an evidence-based optimization approach. This allows the model to better adapt to the characteristics of the corpus. Similarly, the model dynamically adjusts eta values during training to determine the optimal word-to-topic assignment. This enables topics to better reflect the statistical relationships within the corpus. Additionally, we set LDA to iterate 15 times over the entire corpus, which improves convergence and allows for a more stable training of topics. We also set the model to perform 100 training iterations per document, ensuring a more accurate optimization of topic assignments. In the Python script, we set the `random_state = 42` variable to ensure the reproducibility of the results between runs. In turn, we set the variable `chunksize = 256` to control how many documents are processed at a time, balancing the memory efficiency and training stability. This setting makes the model training efficient and better aligned with the dataset size. We also applied a cosine similarity-based filter to identify and retain only those keywords that are semantically related to specific security terms.

We used stop words derived from the NLTK library⁶ and stop-word lists sourced from other open-access repositories generated by other researchers.⁷ However, in order to effectively apply the LDA algorithm, we found it necessary to pre-process it further. Despite training the model, certain words appeared frequently in the topics without contributing semantic value. Consequently, we executed the model iteratively until we identified words that did not add value to

² <https://www.cve.org>.

³ <https://cwe.mitre.org>.

⁴ <https://capec.mitre.org>.

⁵ <https://radimrehurek.com/gensim/>.

⁶ <https://www.nltk.org>.

⁷ <https://github.com/Alir3z4/stop-words>.

the topics and subsequently classified them as stopwords. We have implemented the model with 5, 10, 15, 20, 25, 30, 35, 40, 45, and 50 topics until identifying these words that do not add value. To select the optimal number of topics, we analyzed the values of perplexity and coherence (Hasan et al., 2021). Perplexity measures how well the model predicts a dataset based on the log-likelihood of the corpus; lower values indicate better predictive ability but not necessarily better semantic interpretation. The perplexity is given by the following formula:

$$\text{Perplexity}(D) = \exp\left(-\frac{\sum_{d=1}^{|D|} \log P(w_d)}{\sum_{d=1}^{|D|} N_d}\right)$$

where D is the set of documents, w_d are words in the document, $P(w_d)$ is the probability of the document according to the model, and N_d is the number of words in the document. On the other hand, coherence evaluates the semantic consistency between the most representative words of each topic, which reflects how interpretable and understandable topics are to humans. The coherence is described as follows:

$$\text{Coherence}(W) = \sum_{i=2}^N \sum_{j=1}^{i-1} \log\left(\frac{D(w_i, w_j) + \epsilon}{D(w_j)}\right)$$

where $D(w_i, w_j)$ is the number of documents where w_i and w_j co-occur, $D(w_j)$ is the number of documents where w_j emerge and ϵ is the smoothing constant (to avoid $\log(0)$). Both metrics are instrumental in determining the appropriate number of topics and assessing the efficacy of the LDA model. A lower perplexity indicates a superior model, while a higher coherence score reflects enhanced clarity and interpretability of the topics.

To evaluate the consistency of the topics, we calculated Cohen's kappa (Sun, 2011) value with two researchers. The value is calculated with the following formula:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o is the proportion of observed agreement and P_e is the proportion of expected agreement by chance. Cohen's Kappa applied to LDA serves as a measure of the interpretive quality of topics. This allows us to validate whether the model results are usable and understandable to humans.

3.5. Data analysis

In order to identify the topics with respect to RQ1, we analyzed the terms generated by the LDA model. For each topic obtained by the model, we studied in detail the context of each term and, by consensus, determined the name for each term in order to classify the detected topics. In addition, we selected an issue that represents the essence of the topic and discussed it to complement the analysis. In the process of selecting a name for each topic, each participant proposes a name that encapsulates the essence of the topic. Subsequently, these proposed names are presented, and if consensus is achieved, the name is finalized. In the absence of agreement, the reasons for the discrepancies are deliberated, and the discussion is reiterated until a consensus is reached.

Regarding RQ2, we conducted an open search query utilizing the terms "quantum" and "software" within the Common Vulnerabilities and Exposures databases. This search yielded 136 vulnerabilities pertinent to the query. Subsequently, we examined each identified topic to analyze the issues and map any potential vulnerabilities.

About RQ3, we identified the questions asked by developers on GitHub. Subsequently, we grouped and classified them into "what?", "how?", "why?", "for what?" and other. We used descriptive statistics to discuss results.

Table 2

Values of perplexity and coherence for each topic.

Topics	Coherence	Perplexity
5	0.462777	-8.097179
10	0.430697	-8.186538
15	0.473141	-8.270300
20	0.445569	-8.952740
25	0.410012	-9.185786
30	0.445978	-9.431596
35	0.411506	-9.686944
40	0.394388	-9.912554
45	0.423626	-10.132021
50	0.403009	-10.374812

4. Results

Table 2 presents the values obtained for both perplexity and coherence for each topic. Based on the results, we identified 15 topics as optimal, as the LDA model exhibited the highest coherence score (0.4731), indicating superior interpretability and semantic consistency in the generated topics. Although perplexity generally decreases with an increase in the number of topics, as evidenced by lower values for 20 or more topics, this reduction does not necessarily represent enhanced thematic quality, as it may indicate overfitting. In this instance, the 15-topic model achieved a balance, maintaining robust generalization without compromising interpretability. outlines the topics derived from our research, along with the principal terms associated with each topic.

The result of Cohen's kappa value is 0.545, which indicates that the two researchers agree on the assignment of topics in several issues, implying that there is adequate semantic coherence, allowing a relatively stable interpretation, but a certain degree of ambiguity still persists. This implies that there may be some topics with semantic overlap, or that are difficult for researchers to distinguish. Finally, issues in which there was no agreement on topic assignment were resolved by consensus among all authors of the study.

Id	Topic	Terms
T1	Backend configuration	Initial, supports, backend, checked, polynomial, b, modes, configured, qnodes, p
T2	Code failure	Elements, layer, vulnerability, repository, coefficients, written, layers, subroutines, security, failed
T3	Spatial optimization	Space, optimization, docstring, slower, symmetry, lattice, cell, dear, electronic, wires
T4	Parallelization dependencies	Threads, cu, potential, dependencies, mpi, report, ecp, installations, walltime, optimal
T5	Authentication of quantum operations	Device, differentiation, op, authentication, specialunitary, calculation, capabilities, clear, operations, execution
T6	Transformations in quantum evolutions	Matrix, block, transformation, group, evolution, u, transforms, dummy, bug, developers
T7	Compilation of quantum circuits	Cell, structure, primitive, signature, controlledqubitunitary, response, instructions, schedule, grid, compile

T8	Improving quantum algorithms arithmetic	Adder, large, improved, ideas, gates, qft, require, modified, encoded, incrementer
T9	Verification	Increases, tuple, returned, returns, trust, manually, turns, identify, leading, systems
T10	Response modeling	Response, extrapolation, constant, lattice, included, addition, section, top, calculations, nnp
T11	Noise testing	mpi, patch, rx, runs, regression, embedding, shows, separate, variables, noise
T12	Timing optimization	Read, dictionary, entries, higher, reporting, settings, optimization, noticed, timing, backend
T13	Parameterized circuits	Circuit, generator, installation, gates, operator, response, zne, circuits, parametrized, coefficient
T14	Modular validation	Node, nodes, submodule, block, validation, timing, psmpp, declaration, discussions, errors
T15	Development workflow	Patch, decomposition, merge, fixes, bug, pipeline, development, instances, date, minor

4.1. RQ1: Topics identified

T1: Backend configuration. The terminology associated with T1 indicates that the papers categorized under this topic explore the essential technical configurations necessary for the execution of quantum software. For instance, the terms “initial” and “configured”, when contextualized within the issues related to T1, suggest that developers concentrate on the preliminary phase of the process, potentially focusing on configuring the quantum execution backend, whether it be a simulator or actual hardware (pennylane_open_2, cp2k_closed_11). Additionally, terms such as “qnodes” and “modes” imply that developers set up quantum nodes and operations prior to execution. This suggests that these studies are dedicated to establishing the requisite conditions for quantum operations to be conducted effectively and reproducibly. The term “polynomial” is associated with configurations pertaining to anticipated computational complexities (strawberryfields_closed_55, qiskit_closed_853).

T2: Code failure. The terminology associated with T2 indicates concerns regarding security issues within the software development life cycle. The explicit use of terms such as “vulnerability” and “repository” suggests that the issues addressed pertain to the identification, description, or resolution of potential vulnerabilities, particularly those related to the structure or configuration of source code stored in repositories (Cirq_closed_593, ProjectQ_open_6). Additionally, terms like “failed” and “complaint” highlight execution errors or problems associated with poorly defined or faulty subroutines, which may lead to implementation errors or information leaks. Consequently, the topic encompasses a combination of technical components (such as subroutines, layers, and repositories) and their associated risks in terms of failures or security breaches.

T3: Spatial optimization. The parameters of this topic facilitate the identification of a significant association with physical simulations or the modeling of materials within quantum contexts. Such simulations frequently necessitate the optimization of spatial structures, including unit cells and crystalline symmetries. In this context, terms such as “docstrings” and “slower” pertain to performance or documentation issues in codes associated with these models (Cirq_closed_1732, cp2k_closed_332). Additionally, terms like “wires” and “space” imply physical or topological representations, potentially within a circuit or lattice framework (pennylane_closed_314). Overall, this topic appears to concentrate on representation and optimization challenges in systems that model quantum physical structures, such as materials or spatially organized qubit networks.

T4: Parallelization dependencies. This topic denotes concepts prevalent in parallel execution environments, particularly within high-performance computing (HPC) systems. For instance, the Message Passing Interface (MPI) serves as a standard technology for distributed computing, and its reference suggests that the papers under discussion address the execution of quantum software in parallel environments (QUICK_open_11). Terminology such as “threads” and “walltime” pertains to the total execution time utilized by a process (cp2k_open_11, strawberryfields_open_10). Additionally, “potential” and “ecp” represent specific dependencies on physical libraries or projects, such as the Exascale Computing Project (Messina, 2017) (cp2k_open_14). Other terms within this topic underscore the focus on the installation and configuration of environments with multiple dependencies, which is essential for complex quantum projects.

T5: Authentication of quantum operations. The terminology associated with this subject primarily pertains to the secure and precise execution of quantum operations on both physical devices and simulators. The concept of authentication implies that certain studies focus on mechanisms for user or system access and validation, which are crucial in shared quantum computing environments. Terms such as “op”, “operations”, and “execution” are directly linked to the implementation of quantum gates or other instructions (Cirq_closed_1611). The reference to “specialunitary” indicates that the operations in question possess specific mathematical properties, such as $SU(n)$ matrices (Nielsen and Chuang, 2010), which are fundamental in quantum algorithms (pennylane_open_132). Furthermore, the terms “calculation” and “differentiation” suggest a functional component aimed at gradient analysis or function optimization within quantum circuits (pennylane_closed_264).

T6: Transformations in quantum evolutions. The terminology employed in this discussion suggests that the issues address the manipulation of mathematical structures, specifically matrices, which are crucial for representing quantum gates and operators. The use of the terms “transformation”, “group”, and “u” implies unitary or symmetry transformations, which are prevalent in the evolution of quantum states (pennylane_closed_464, pennylane_closed_366). In this context, the term “dummy” pertain to placeholder variables utilized in mathematical functions or codes (qiskit_closed_11, cp2k_closed_384). The mention of bugs is significant, indicating that the transformations under consideration are not solely theoretical but also involve implementation or integration errors. Consequently, this topic encompasses both the conceptual and practical challenges associated with the application of transformations in quantum software.

T7: Compilation of quantum circuits. This topic addresses the structural construction of quantum systems at both physical and logical levels. The term “primitive” refers to the fundamental units of a structure, which may be geometric primitives in material simulations or logical blocks in quantum compilers (cp2k_open_50). The terms “structure” and “grid” pertain to the spatial or architectural representations of these elements (OpenFermion-Cirq_closed_70). The concept of compilation situates the topic within the context of translating logical or physical structures into an executable or interpretable format using a

quantum backend. “Instructions” and “schedules” refer to discussions concerning the planning and sequencing of tasks within an execution flow (qiskit_closed_2832, qiskit_closed_2708). Similarly, the term “signature” may denote validations or definitions of functions or operation frames (nuSQuIDS_open_6).

T8: Improving quantum algorithms arithmetic. This topic presents words such as “adder”, “qft” (Quantum Fourier Transform (Nielsen and Chuang, 2010)), “incrementer”, “encoded”, and “gates”, which clearly indicate the implementation or improvement of quantum algorithms with arithmetic functions (qiskit_closed_2519). QFT is a key operation in many quantum algorithms that require periodic transformations; therefore, its inclusion emphasizes a technical approach. Encoded suggests the use of specific representations of information, while modified, large, and ideas indicate proposals for extension or innovation. Terms such as “adder” and “incrementer” address the encoding of addition operations, which is common in algorithms such as Shor and modified Grover (Nielsen and Chuang, 2010) (pennylane_open_28).

T9: Verification. The concepts associated with this subject suggests an emphasis on the manual validation of results, their interpretation, and the confidence in the quantum systems that produce them. Generally, the terms indicate an analysis centered on the output of functions, specifically what a quantum system yields following the execution of a process. The term “manually” denotes human intervention or verification, due to the uncertainties inherent in automation (qiskit_closed_76, pennylane_closed_543). The terms “identify” and “lead” are related to diagnostic tasks or the detection of issues within the results or behaviors of a system (pennylane_closed_679). Consequently, this topic is oriented towards the confidence in the outputs generated by quantum software and the mechanisms, whether automatic or manual, through which developers review, validate, or interpret these results.

T10: Response modeling. This topic is related to the computational modeling of physical systems. The concepts of response and extrapolation are discussed in contexts where the physical reactions of materials or systems to external stimuli are analyzed, and the findings are generalized or projected. Lattice and Neural Network Potentials (Kocer et al., 2022) (“nnp”) refer to network structures at both the physical and machine learning levels, which are employed to model interatomic potentials (cp2k_open_105). The remaining terms denote specific parameters or contributions that are integrated into a comprehensive model. In this context, the topic encompasses research where complex calculations are conducted to represent the responses of quantum or material systems, necessitating the projection of these data beyond the initial dataset, a common practice in molecular or physical simulations.

T11: Noise. This topic encompasses terms such as “mpi”, “patch”, “rx”, “runs”, “regression”, “noise”, and “embedding”, which pertain to the testing and maintenance of quantum software within complex and noisy environments. The terms “patch” and “regression” refer to software maintenance processes, specifically the rectification of bugs without the introduction of new errors (qiskit_open_800, qiskit_closed_2355). The term “rx” denotes X -rotated quantum gates (qiskit_open_83). “Noise” and “embedding” are pertinent to quantum simulation challenges: the former indicates the evaluation of noise effects, while the latter pertains to the mapping of problems onto actual quantum architectures (qiskit_open_327).

T12: Timing optimization. The topic encompasses terms that signify the examination of operational enhancements pertaining to backend systems and software runtimes. The terms “read”, “dictionary”, and “entries” denote developer discussions focused on the analysis of internal data structures, such as Python dictionaries, which may be subject to optimization or auditing (qiskit_closed_1188). The issues concerning “settings” and “optimization” are pertinent to the accurate configuration of quantum environments, as backends — whether simulators or actual hardware — necessitate specific parameters to achieve optimal performance. The term “timing” implies the assessment of timing performance to facilitate the comparison of various strategies (qiskit_open_15).

T13: Parameterized circuits. Issues focused on the development and evaluation of quantum circuits, with a particular emphasis on those that are parameterizable. The terms “generator” and “parametrized” denote templates or functions that facilitate the construction of adaptable circuits for a variety of problems (qiskit_open_771). The concepts of “gates” and “operator” underscore the functional components integral to these circuits (qiskit_open_866). Zero Noise Extrapolation (Giurgica-Tiron et al., 2020) (“zne”) is a specific technique employed to mitigate noise in quantum computing, indicating that the circuits generated are also assessed in noisy environments (pennylane_closed_56). The term “response” is referenced as a potential output of the circuit under specific conditions, while “coefficient” likely pertains to the adjustable values within the objective functions (pennylane_closed_938).

T14: Modular development. The terms associated with T14 exemplifies a modular framework, a characteristic feature of extensive software projects where submodules are employed to manage distinct segments of the software repository. The term “node” denotes both the elements of the computational architecture and the functional units of the circuit (pennylane_closed_788). The concepts of validation and errors are frequently referenced by developers when discussing issues pertaining to the execution of tests aimed at verifying system integrity and identifying potential errors during execution. The term “timing” underscores the measurement of time as a criterion for assessing performance (pennylane_closed_618).

T15: Development workflow. This topic examines the software life cycle activities associated with maintenance, correction, and incremental enhancements. The terms “patch” and “fixes” are frequently referenced in discussions concerning modifications implemented to address specific software bugs (qiskit_open_550). The term “bug” indicates that these modifications are responses to defects within the code. The concepts of “pipeline” and “merge” are characteristic of collaborative workflows on platforms where the validation of changes is automated through Continuous Integration and Continuous Deployment (CI/CD) processes (qiskit_open_102, pennylane_closed_543). “Decomposition” pertains to the refactoring of complex functionalities into more manageable components (pennylane_closed_366).

Key findings of RQ1:

- The topics concentrate on identifying deficiencies at the layer, subroutine, and repository levels, demonstrating a clear understanding of vulnerability exposure.
- Developers have expressed significant interest in implementing controlled access mechanisms prior to the execution of quantum operations.
- Developers consistently express concern regarding the manual verification of results, the modular validation of the system, and the system’s tolerance to failures induced by physical or logical noise.

4.2. RQ2: Security concerns

Seven topics correspond to the security concerns delineated in the CVEs: T2, T5, T6, T10, T11, and T15 (See Fig. 2).

The primary concept defining T2 is code failure. Our analysis reveals that CVE-2025-21647, associated with logic errors and boundary checks in kernel structures, conceptually aligns with discussions on programming flaws in controllers and fairness verification, as evidenced in qiskit_closed_2643. Similarly, CVE-2012-1842, which describes a cross-site scripting (XSS) vulnerability in embedded web interfaces, relates to developer comments in pennylane_closed_278, highlighting concerns over input sanitization and frontend component errors. CVE-2022-1115, a heap-buffer-overflow flaw in the PushShortPixel() function of quantum-private.h, is discussed in the context of

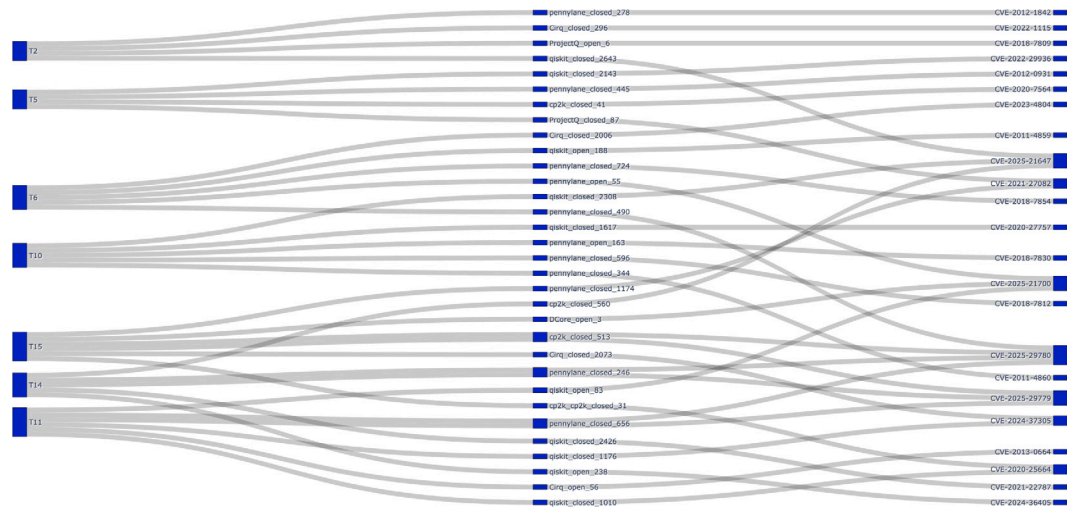


Fig. 2. Relationship between topics, issues and vulnerabilities detected.

virtual memory management in `Cirq_closed_296`, where developers address overflow conditions. Furthermore, CVE-2018-7809, concerning unverified password changes in embedded servers, aligns with discussions in `ProjectQ_open_6`, where authentication flaws in critical configuration changes are scrutinized. CVE-2019-6852, linked to information exposure in quantum controllers, resonates with multiple discussions mentioning risks from excessive logging or exposure to environmental variables.

In the context of T5, we have identified vulnerabilities pertaining to access control, authentication, remote code execution, and insecure memory handling, which are issues that also manifest in developer-related concerns. For example, CVE-2022-29936, which facilitates remote code execution through unsafe deserialization, aligns with the discussions in `qiskit_closed_2143`, where the dangers of loading unvalidated data are examined. Similarly, CVE-2012-0931, which underscores the absence of authentication between software and hardware in quantum controllers, mirrors the discussions on authentication flaws in `pennylane_closed_445`. Concerning buffer overflows, CVE-2020-7564 is related to the discussions in `cp2k_closed_41`, where developers address vulnerabilities arising from uncontrolled inputs. Implicit references to CVE-2021-27082, associated with remote execution in development environments, were also identified in `ProjectQ_closed_87`, where developers express concerns regarding isolation and execution environment security. These correlations indicate that developers, although not explicitly citing CVEs, actively acknowledge and discuss critical security issues that are subsequently formalized in databases such as MITRE.

In relation to T6, CVE-2023-4804 is related to `Cirq_closed_2006`, where developers deliberate on methods to conceal or restrict debug modes that could be inadvertently activated by unauthorized users. Likewise, CVE-2011-4859, which addresses unauthorized access to quantum network modules via Ethernet vulnerabilities, parallels concerns in `qiskit_open_188` regarding port exposure and insufficient control over network interfaces. Additionally, CVE-2018-7854, which involves uncaught exceptions leading to denial of service, aligns with the issues discussed in `pennylane_closed_724`, where unhandled errors are demonstrated to disrupt critical quantum system processes. CVE-2025-21700, associated with memory corruption and the misuse of scheduling structures, is analogous to the discussions in `pennylane_open_55` concerning unsafe execution conditions during asynchronous operations. Finally, CVE-2025-29780, which reveals vulnerabilities in Feldman's VSS cryptographic verification (Chen and Lindell, 2024) library in Python, is conceptually linked to `pennylane_closed_490`, where developers debate the use of constant-time comparisons and the implementation of sensitive functions in Python versus more secure alternatives such as C or Rust.

T10 reports identified common security concerns related to mathematical computations, HTTP header vulnerabilities, and information exposure. For instance, CVE-2020-27757, which details a value overflow issue in floating-point mathematical functions, corresponds to `qiskit_closed_1617`, where out-of-range values during resizing operations compromise result reliability. Similarly, CVE-2018-7830, a CRLF sequence neutralization flaw leading to HTTP response splitting, parallels issues in `pennylane_open_163` concerning insecure header management and the necessity for response sanitization. Furthermore, CVE-2018-7812, which describes information exposure through inconsistent server responses, is mirrored in `pennylane_closed_596`, where developers address the risk of system errors revealing internal execution details. CVE-2011-4860, which highlights unsafe password generation, is related to `pennylane_closed_344`, which questions deterministic credential generation. Lastly, CVE-2025-21647, which underscores fairness counter boundary check issues in the kernel, conceptually aligns with observations in `qiskit_closed_2308` regarding the validation of queues and asynchronous scheduling structures.

Concerning T11, several shared security concerns have been identified, particularly in relation to memory management, post-quantum cryptography, and insecure communications. CVE-2024-37305, which outlines vulnerabilities in the oqs-provider (OpenSSL post-quantum algorithm implementation (Butin et al., 2017)), corresponds with `qiskit_closed_1176`, where developers discuss the challenges of securely integrating new cryptographic algorithms, including issues of interoperability and validation. CVE-2025-21700, addressing improper access in Linux kernel structures, is related to `qiskit_open_83`, where developers report synchronization issues and vulnerabilities in shared structures within parallel environments. CVE-2025-29779 and CVE-2025-29780, associated with information leaks in Feldman's VSS implementation in Python, are thematically linked to `pennylane_closed_656`, where the use of cryptographic functions without constant-time execution or low-level isolation is scrutinized. The discussion explicitly highlights the risks of statistical inference and timing attacks. Furthermore, the critical CVE-2013-0664, associated with remote command execution via Modbus messages in Quantum PLCs, connects to `Cirq_open_56`, where concerns are raised regarding command validation in quantum network protocols and the necessity to verify headers and malicious commands in persistent connections. CVE-2020-25664, related to out-of-bounds writes in `WriteOnePNGImage()`, corresponds to `qiskit_closed_1010`, where developers detail memory mishandling errors during visualization, particularly when processing binary structures and images.

In relation to T14, shared concerns have been identified regarding input validation, control flow, and post-quantum cryptography. CVE-2021-22787, which pertains to improper input validation that facilitates Denial of Service (DoS) attacks, is documented in `qiskit_closed_2426`, where developers discuss unexpected inputs causing system component failures. CVE-2025-21647, associated with logic errors in fairness counters, is analogous to comments in `cp2k_closed_560` concerning race conditions and the absence of explicit limits in parallel processes. CVE-2025-29779 and CVE-2025-29780, which relate to execution and security issues in the Python implementation of Feldman's VSS, correspond to discussions in `pennylane_closed_246`, where risks are highlighted regarding reliance on third-party cryptographic libraries without comprehensive source code validation. The same file also references design decisions that may facilitate side-channel attacks. CVE-2024-36405, which reports a timing leak in the `liboqs` library, is connected to `qiskit_open_238`, where developers are cautioned about timing discrepancies in post-quantum algorithms that could be exploited if constant-time execution is not enforced.

Regarding T15, a significant vulnerability is CVE-2021-27082, which pertains to remote code execution within the Quantum Development Kit for Visual Studio Code. This vulnerability is discussed in `pennylane_closed_1174`, where developers address execution environment risks and emphasize the necessity of properly isolating extensions. Similarly, CVE-2024-37305, which involves flaws in the `oqs-provider` and its integration with OpenSSL 3 for post-quantum cryptography, corresponds with `Cirq_closed_2073`, where concerns are raised about interoperability between cryptographic libraries and the occurrence of silent errors during validation. Additionally, CVE-2025-29779 and CVE-2025-29780, which are associated with leaks and unsafe execution in the Python implementation of Feldman's VSS, find correlations in `cp2k_closed_513`, which discusses the vulnerabilities of high-level cryptographic libraries and the imperative for constant-time validation. CVE-2025-21700, which impacts the Linux kernel through improper scheduling structure usage, aligns with `DCore_open_3`, which describes analogous synchronization issues and potential race conditions in parallel execution tasks. CVE-2020-25664, concerning an out-of-bounds memory write during image processing, is consistent with discussions in `cp2k_closed_31`, which analyze how visualization errors can undermine the stability of graphical modules.

Key findings of RQ2:

- Topics and CVEs highlight unverified password changes and missing authentication between software–hardware components.
- Most CVEs and issues agree that heap-buffer overflows, out-of-bound writes, and input sanitization flaws persist in quantum software.

4.3. RQ3: Type of question

Fig. 3 describe the distribution related to type of questions obtained from the issues.

The prevalence of “how” questions in topics such as T3, T5, T6, and T11 underscores a pronounced emphasis on operational and technical dimensions. In the context of quantum software, it is observed that these inquiries frequently pertain to issues concerning the secure integration of algorithms, quantum error management, result verification, and the safeguarding of hybrid communication channels (quantum–classical).

Conversely, topics characterized by a substantial number of “what” questions (e.g., T4, T11, and T14) often indicate preliminary concerns or the necessity for conceptual elucidation. In these instances, it has been observed that developers seek to comprehend the available functionalities, identify potential vulnerabilities, and ascertain which

practices are deemed secure within quantum environments. Furthermore, “what” questions highlight deficiencies in documentation or a lack of transparency within development platforms. We realized that this small set of questions points to issues that arise when entering new algorithms at the time of sig-in in quantum applications. According to what we observed in the developers' discussion, there is interest in knowing what consequences may occur when modifying authentication mechanisms in different contexts, for example, in a new operating system, in software testing, and in the performance of quantum software.

Although less frequent, “why” questions are particularly pertinent for identifying design flaws, inconsistencies in system behavior, or architectural decisions that may compromise security. Topics such as T6, T7, and T8 encompass this type of inquiry, which can serve as early indicators of issues related to internal logic, misinterpretations in security protocols, or inadequate key and encryption management practices.

The “for what” question type, though less frequently encountered, possesses significant interpretive value as it typically pertains to the intention or purpose underlying certain functionalities. For example, in topics T7, T12, and T13, such inquiries facilitate the identification of issues at the intersection of functional design and security objectives. Comprehending the “for what” aspect of a function's execution can elucidate whether principles such as least privilege, separation of duties, or the risk of introducing unintended backdoors are being implemented. These questions typically arise when expectations of secure behavior are unmet, potentially leading to the exposure of critical vulnerabilities or a decline in system reliability. In type of question, code-related questions focus on the integration of quantum algorithms and code libraries into quantum programming languages, primarily Qiskit and Q#. Such questions open the debate as to why the compilation of quantum software differs significantly when library changes are made. However, questions targeting cryptography mostly discuss why certain cryptographic algorithms do not work in specific environments. For example, many developers question why some Qiskit libraries do not operate on Windows operating systems. In this regard, developers have detected shortcomings in the Kyber and P-256 elliptic-curve algorithms.

We also categorized certain questions as “other” (189 in total), which pertain to issues encompassing technical statements, error reports, or those lacking an explicit interrogative structure. Nonetheless, we opted to investigate these issues due to their potential to harbor implicit security concerns, particularly within error descriptions, change requests, and discussions regarding external dependencies.

Key findings of RQ3:

- Questions related to “How?” describe relevant information about what developers discuss regarding security in quantum software projects.
- The high frequency of “how” questions reveals a strong concern for the practical implementation of safe quantum components.

5. Discussion

The analysis of the 15 topics (RQ1) has identified a comprehensive array of critical areas encompassing the entire lifecycle of quantum and classic software development. Certain topics concentrate on the initial phases, such as parameter configuration and backend selection, underscoring the significance of pre-execution decisions for quantum tasks. Other technical components, including subroutines and software layers, address concerns related to structural failures and security vulnerabilities. There is a pronounced focus on the representation and optimization of physical systems, particularly qubit networks and spatially modeled materials. From an operational standpoint, the analysis underscores issues associated with the installation and efficient

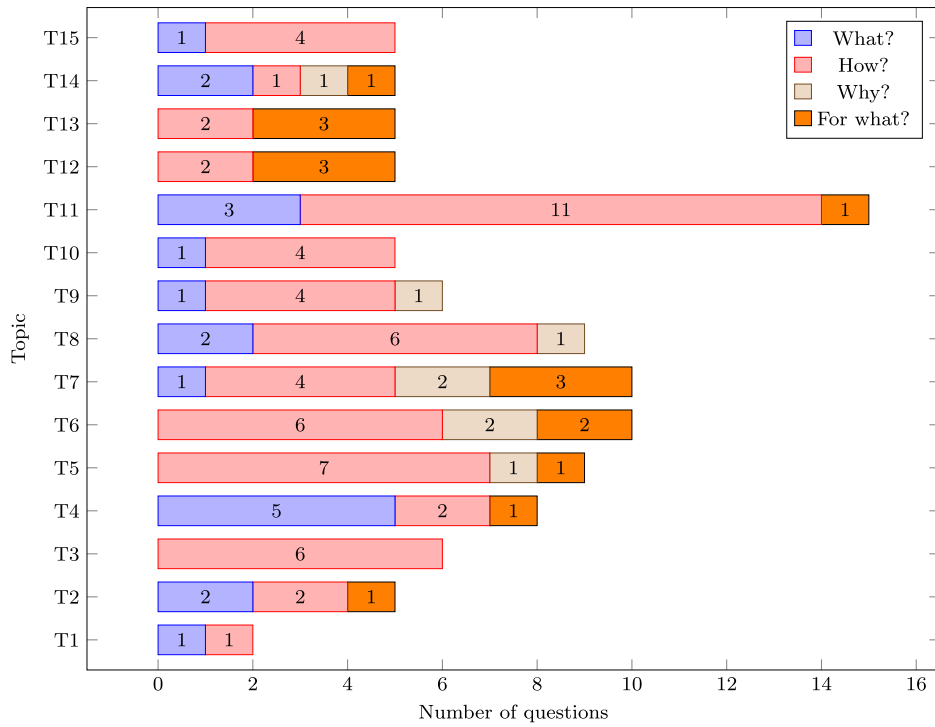


Fig. 3. Distribution of types of questions per topic.

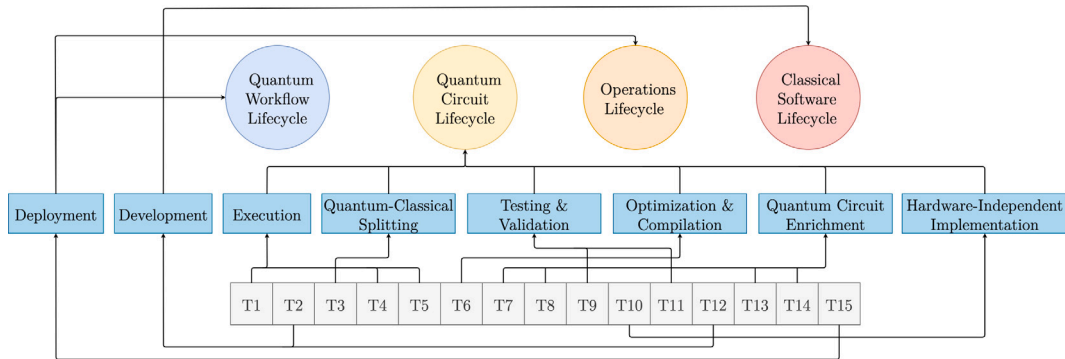


Fig. 4. Relationship of the identified topics with life cycle phases associated with quantum workflow, quantum circuits, operations, and classic software.

execution of quantum software on parallel platforms, which necessitate technical precision and secure authentication protocols. Several topics underscore the necessity of reliably transforming and evolving quantum states, incorporating tasks such as debugging, verification, and error-handling. Similarly, the role of compilers and simulators in translating physical structures into executable instructions is highlighted. Quantum software has emerged as a pivotal tool for extrapolating physical properties, validating hypotheses, and simulating experimental conditions. This perspective is further enriched by topics centered on regression testing, noise control, and patch management. Enhancing backend performance and mitigating noise in dynamic circuits remain ongoing concerns for runtime efficiency. Furthermore, the analysis identifies issues related to the validation of isolated components and the collaborative, continuous maintenance of software, which are crucial for the stability, scalability, and security of quantum ecosystems.

To contextualize the relationship between the topics and lifecycle of quantum and classical software development, we compared the topics to the lifecycle phases proposed by [Weder et al. \(2022\)](#), which describes four types of lifecycles, which are: quantum workflow, quantum circuit, operations and classical software (see Fig. 4).

In our analysis, we observed that the vast majority of topics align closely with the definitions of the phases of the quantum circuit lifecycle. In this lifecycle, the phases execution, quantum-classical splitting, testing & validation, optimization & compilation, quantum circuit enrichment, and hardware-independent implementation show strong similarities with the identified topics. The execution phase refers to all steps required to run quantum circuits. In this context, topics T1, T4, and T5 addressed various security concerns that focused on this phase. In most situations, quantum circuits must be compiled across different stages, and topics related to the execution phase reveal developer concerns from the perspective of configuration, dependency management, and authentication levels required to compile quantum operators. The optimization & compilation phase is also one in which developers express concerns (T6). In this sense, the concerns focus on validating that matrix transformations of quantum states do not compromise data integrity. Regarding the quantum-classical splitting phase, T3 focuses on security concerns in the context of simulation optimization. The security concerns represented by T7, T8, T13, and T14 correspond to the quantum circuit enrichment phase. This phase primarily addresses the preparation of quantum circuit states. This involves different encodings that, in some cases, raise security concerns

when implementing or loading external libraries to run the quantum circuit. T9 and T11 discuss the security concerns that align with the testing and validation phases. Both topics address the verification and noise aspects, which are relevant when testing quantum circuits. T10 corresponds to the hardware-related phase, as this topic directly addresses the modeling of physical systems. The remaining topics (T2, T12, and T15) are related to the development and deployment phases associated with the lifecycles of quantum workflows, operations, and classical software.

The insights provided by CVEs concerning the identified issues (RQ2) enhance the comprehension of the security concerns disclosed in these issues. It is anticipated that not all issues can be directly associated with CVEs. However, 32 issues, representing 11% of the total, demonstrated a correlation with CVE. Although this number is not substantial, it underscores the security aspects in quantum software that are pervasive across various domains of software engineering. Summarizing the CVE descriptions presented in Fig. 2, these vulnerabilities encompass buffer overflows, improper input validation, insecure deserialization, race conditions, remote code execution, and authentication/authorization failures. These concerns are particularly critical in hybrid quantum–classical environments, where software frameworks (such as Qiskit, Cirq, or Braket SDK) interact with specialized hardware and classical control systems. Based on discussions among quantum software developers, it is evident that rigorous validation of quantum circuit parameters and hardware configurations is imperative, as malicious inputs can compromise both algorithm execution and the security of cloud-based quantum platforms. Furthermore, the mapping between issues and CVEs reveals that middleware orchestrating quantum execution frequently relies on object serialization, which introduces potential attack vectors if not properly controlled. In this context, quantum simulators optimized in C/C++ also inherit classic risks such as buffer overflows and race conditions.

The correlation between issues and CVE vulnerability records does not yet yield sufficient knowledge or a comprehensive characterization of the primary security concerns in quantum software development. Nevertheless, this preliminary analysis provides valuable insights into emerging trends and potential security challenges that may become critical as quantum ecosystems evolve. The observed concerns, such as memory integrity issues, input validation weaknesses, weak authentication, and risks associated with serialization and remote code execution, indicate that the community must enhance its secure design, testing, and quality assurance strategies. Although the current CVEs predominantly originate from classical software, the identified issues underscore aspects that could be transferred to or amplified within hybrid quantum–classical contexts. Consequently, these findings serve as an early warning to anticipate the specific security controls that must be addressed in the future development of quantum software.

The analysis of question types (RQ3), particularly “how” questions, underscores the necessity for more precise guidelines on configuring and deploying quantum systems that adhere to security and robustness requirements, especially within hybrid environments (classical–quantum). Similarly, “what” questions highlight a significant concern regarding the current lack of clarity about the actual capabilities of quantum libraries in terms of security. Furthermore, “why” questions are directly associated with potentially disruptive events from a security perspective, such as errors in intermediate measurement, consistency failures in quantum compilers, or confusion in authentication flows between distributed components. Additionally, “for what” questions prompt reflection on the ambiguity in the functionalities of quantum software, as this lack of clarity can lead to misuse or poor practices in complex quantum architectures.

5.1. Practical implications

Topics concerning backend configuration, parameters, and internal structures underscore the critical importance of establishing consistent

and secure environments prior to the execution of quantum algorithms, which significantly influences the reproducibility and efficiency of experimental outcomes. Discussions related to subroutines, abstraction layers, and compilers reflect the intricate nature of modular design and the necessity of maintaining stable, scalable, and robust interfaces, while considering potential risks such as memory failures and execution vulnerabilities. Similarly, topics associated with the representation of quantum physical structures, including qubit networks or simulated materials, illustrate how software facilitates scientific exploration and necessitates specialized tools for precise modeling and optimization.

Challenges associated with the efficient execution on parallel architectures highlight issues in concurrency management, synchronization, and authentication, which are essential for integration with actual hardware. Processes such as verification, regression testing, and debugging are vital components that must be incorporated into the continuous lifecycle of quantum software. The management of noise, error mitigation, and the evolution of dynamic states necessitate automated and fault-tolerant mechanisms. The validation of individual components also impacts software quality, particularly in collaborative and open environments. The emphasis on maintenance and collaboration indicates that the sustainability of quantum software development depends on adapting traditional software engineering practices to this emerging paradigm, including version control, dependency management, and security analysis.

5.2. Limitations

The constraints of our research include that the chosen projects only comprise a restricted range of security concerns in quantum software development, and the selected issues are limited to encompassing all security concerns. For the selected projects, we executed an extensive search protocol to distinguish the most illustrative quantum software initiatives. We discovered projects focused on various objectives but aimed at quantum software development. Nevertheless, we cannot guarantee that we have acquired the most substantial collection of projects, but we can affirm that the selected projects adhere to the stringent inclusion and exclusion criteria, and furthermore, Table 1 serves as an initial precedent for a dataset of quantum projects.

The interpretability of topics can be examined from multiple perspectives. In this study, we concentrate on identifying the topics (RQ1), examining the security concerns articulated by the topics based on CVEs (RQ2), and analyzing the types of questions posed by developers in the issues (RQ3). While these insights may not necessarily be indicative of security in quantum software projects, our research establishes the initial precedent on the security issues that software developers discuss in software projects. This initial precedent can serve as a foundation for further investigation into classical software development techniques that may be applicable in the emerging field of quantum software engineering.

6. Software engineering perspective

We identified efforts to apply software development practices to quantum software projects. Some of these projects use file organization methodologies and code dependencies that describe a tentative architecture. However, the development of the quantum software projects we analyzed appears to be imperative. Upon analyzing the codes of quantum software projects, we found that some projects focus on specifying in detail how algorithms should be performed and how data should be manipulated in a code that translates into a sequence of instructions that modify the state of the program and the data stored in memory. As we analyze the results of our research, much of the developer discussion points to software architecture concerns and coincides with what Murillo et al. (2024) describe in their analysis of the challenges in Quantum Software Engineering. Khan et al. (2023) argued that there is a need to describe and provide new techniques and methods for quantum software development. Based on this requirement, in the following subsections, we describe the challenges we have detected in our study.

6.1. Modularity

Modularity facilitates flexibility, maintainability, and scalability of complex systems. Through the division of architecture into independent components, such as data serialization, quantum circuit management, and hardware integration layers, modularity enables the development, testing, and optimization of each module without affecting others. This separation of concerns ensures that modifications, such as the adoption of new serialization formats or the integration of advanced qubit allocation algorithms, can be implemented without disruptions. Moreover, modular design enhances security by encapsulating functionalities, thereby mitigating the risk of vulnerabilities propagating across a system.

Based on the analyzed issues in the context of security, we detected that the developers mentioned modularity facilitates interoperability, enabling developers to integrate new tools, libraries, or hardware backends with minimal reengineering. Furthermore, modularity simplifies debugging and improves fault tolerance because isolated modules can be replaced or repaired independently during system failures.

6.2. Resiliency

Dynamic resource management, encompassing real-time monitoring and optimization of GPU and memory usage, mitigates bottlenecks such as “Out of Memory” errors, which can impede computations. Consequently, it is posited that software architecture styles oriented on validation pipelines ensure that errors in operations, such as quantum circuit decomposition or noise modeling, are detected at an early stage, thereby preventing cascading failures. Through the implementation of modularity and abstraction, a robust quantum software design can attenuate the propagation of vulnerabilities, thus ensuring that failures in one component do not compromise the integrity of the entire system.

6.3. Scalability

The incorporation of distributed computing frameworks such as MPI or OpenMP facilitates parallelization, allowing systems to manage large-scale simulations and hybrid quantum–classical workflows across multiple nodes. A modular software architecture enhances scalability by isolating components, such as data serialization, quantum operations, or machine learning integrations, enabling individual layers to expand without disrupting the entire system. Additionally, error-tolerant designs, such as automated retry and rollback mechanisms, maintain system reliability under increasing workloads. By incorporating these principles, scalable quantum architectures can satisfy the evolving demands of both research and industry, ensuring performance, cost efficiency, and adaptability in a rapidly advancing field.

6.4. Interoperability

By adopting open standards such as OpenQASM (Cross et al., 2022) for circuit representation or Quantum Intermediate Representation (QIR) (Luo and Zhao, 2025) for cross-platform compatibility, architectures can ensure that quantum algorithms are portable across different hardware backends and simulators. In the context of security, developers describe middleware layers as playing a crucial role in abstracting hardware-specific details, allowing developers to focus on algorithm design without being tied to a particular vendor’s implementation. Modular APIs with standardized interfaces further enhance interoperability, enabling the integration of external libraries such as TensorFlow or JAX for hybrid quantum–classical workflows. Interoperability also relies on the adoption of universal data formats such as HDF5 or QASM, which facilitate the sharing and reuse of quantum data across systems. Robust error-handling mechanisms across platforms ensure that distributed workflows remain reliable even when transitioning between different environments.

7. Threats to validity

In this section, we address threats to the validity of our study. Inspired by Wohlin et al. (2012), we discuss internal and external threats.

Internal validity refers to the extent to which the conclusions drawn from a given study are warranted. With this in mind, we have identified three potential threats: a bias in project identification, topic extraction, and the security filter based on keywords related to security. To address the first threat, our mitigation plan highlights the fact that the researchers conducted a search and extraction of projects on GitHub, defining their search strings and identifying the projects. Subsequently, we analyzed the results and generated a consensus list. To address the second threat, we pre-identified topics separately, that is, each researcher analyzed the topics obtained and analyzed the results together. Furthermore, we invited computer science master’s students to perform the same exercise to crosscheck the results and discuss the findings. In addressing the third threat, we conducted a comprehensive investigation into the primary sources of information on software security. Our research involved consulting vulnerability reference databases (CVE), scientific literature, and software repositories that specialize in security strategies. Despite these efforts, there remains a possibility that not all keywords relevant to security in software development have been considered. Nevertheless, the selection of keywords is substantiated by authoritative sources within the community.

The concept of external validity is related to the extent to which findings can be applied beyond the specific context in which they were obtained. In light of the nascent state of quantum software engineering, particularly with regards to security aspects, it is conceivable that certain relevant security topics may have been overlooked within the community. Consequently, to facilitate the replication of our work in various settings, we have established a replication package. This would enable the validity of our study’s outcomes to be substantiated.

8. Conclusions

In this paper, we present a study that investigates the security topics and design decisions that are discussed by developers in the context of quantum software projects hosted on GitHub. Specifically, we analyzed a total of 2264 filtered issues, which included both open and closed issues, and found that 294 of these issues were related to security. Using the Latent Dirichlet Allocation algorithm, we identified 15 security topics. The issues identified reveal that developers often emphasized that maintaining quantum-state integrity is crucial, as quantum states can be significantly disrupted by noise and decoherence, leading to unreliable computations. This underscores the necessity for rigorous validation methods, such as ensuring auxiliary qubits return to their initial states after decomposition, which highlights the delicate nature of quantum states. This stands in contrast to classical software systems, where data corruption is not inherently tied to the computational process. In the realm of security concerns, developers indicate that numerous security issues they discuss are linked to data confidentiality, especially quantum resistance techniques, methods, and algorithms. Furthermore, traditional security concerns continue to be present in developer discussions regarding quantum software security. Questions related to issues whose nature is to ask “how?” describes most information about security, specifically, security related to code and cryptography.

In our future research, we aim to concentrate on the description and characterization of security design decisions in order to identify architectural patterns and tactics used in quantum software projects. In this regard, our proposed approach employs machine learning techniques, such as CodeBERT (Feng et al., 2020) and GraphCodeBERT (Guo et al., 2020), to identify code snippets that embody architectural tactics, as well as defining structures that represent architectural patterns in quantum software. Moreover, we plan to devote attention to quality attribute scenarios that are specific to quantum software, with the aim of assisting developers and architects in operationalizing and fulfilling quality attributes in quantum software.

CRedit authorship contribution statement

Gastón Márquez: Conceptualization, Data curation, Formal analysis, Project administration, Validation, Writing – original draft, Writing – review & editing. **Muhammad Waseem:** Formal analysis, Investigation, Validation, Writing – review & editing. **Tommi Mikkonen:** Investigation, Methodology, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partially funded by the Iberoamerican Program for Science and Technology for Development (CYTED) through RIPAISC - Iberoamerican Network for the Advancement of Quantum Software Engineering (525RT0174). We also thank Nicolás Murúa for his valuable contribution to our research.

Data availability

I have shared my data in a link described in the paper.

References

- Aguado, A., Lopez, V., Martinez-Mateo, J., Szyrkowicz, T., Autenrieth, A., Peev, M., Lopez, D., Martin, V., 2017. Hybrid conventional and quantum security for software defined and virtualized networks. *J. Opt. Commun. Netw.* 9 (10), 819–825.
- Ali, S., Yue, T., Abreu, R., 2022. When software engineering meets quantum computing. *Commun. ACM* 65 (4), 84–88.
- Alyami, H., Nadeem, M., Alharbi, A., Alosaimi, W., Ansari, M.T.J., Pandey, D., Kumar, R., Khan, R.A., 2021. The evaluation of software security through quantum computing techniques: A durability perspective. *Appl. Sci.* 11 (24), 11784.
- Arias, D., de Guzmán, I.G.R., Rodríguez, M., Terres, E.B., Sanz, B., de la Puerta, J.G., Pastor, I., Zubillaga, A., Bringas, P.G., 2023. Let's do it right the first time: Survey on security concerns in the way to quantum software engineering. *Neurocomputing* 538, 126199.
- Blei, D.M., Ng, A.Y., Jordan, M.I., 2003. Latent dirichlet allocation. *J. Mach. Learn. Res.* 3 (Jan), 993–1022.
- Butin, D., Wälde, J., Buchmann, J., 2017. Post-quantum authentication in openssl with hash-based signatures. In: 2017 Tenth International Conference on Mobile Computing and Ubiquitous Network. ICMU, IEEE, pp. 1–6.
- Campos, R., Mangaravite, V., Pasquali, A., Jorge, A., Nunes, C., Jatowt, A., 2020. YAKE! keyword extraction from single documents using multiple local features. *Inform. Sci.* 509, 257–289.
- Chen, Y.-H., Lindell, Y., 2024. Feldman's verifiable secret sharing for a dishonest majority. *Cryptol. EPrint Arch.*
- Clerk, A.A., Devoret, M.H., Girvin, S.M., Marquardt, F., Schoelkopf, R.J., 2010. Introduction to quantum noise, measurement, and amplification. *Rev. Modern Phys.* 82 (2), 1155–1208.
- Cross, A., Javadi-Abhari, A., Alexander, T., De Beaudrap, N., Bishop, L.S., Heidel, S., Ryan, C.A., Sivarajah, P., Smolin, J., Gambetta, J.M., et al., 2022. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Trans. Quantum Comput.* 3 (3), 1–50.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Fujdiak, R., Mlynek, P., Mrnustik, P., Barabas, M., Blazek, P., Borcik, F., Misurec, J., 2019. Managing the secure software development. In: 2019 10th IFIP International Conference on New Technologies, Mobility and Security. NTMS, IEEE, pp. 1–4.
- Gisin, N., Ribordy, G., Tittel, W., Zbinden, H., 2002. Quantum cryptography. *Rev. Modern Phys.* 74 (1), 145.
- Giurgica-Tiron, T., Hindy, Y., LaRose, R., Mari, A., Zeng, W.J., 2020. Digital zero noise extrapolation for quantum error mitigation. In: 2020 IEEE International Conference on Quantum Computing and Engineering. QCE, IEEE, pp. 306–316.
- Grootendorst, M., 2020. KeyBERT: Minimal keyword extraction with BERT. <http://dx.doi.org/10.5281/zenodo.4461265>.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*.
- Hasan, M., Rahman, A., Karim, M.R., Khan, M.S.I., Islam, M.J., 2021. Normalized approach to find optimal number of topics in latent Dirichlet allocation (LDA). In: Proceedings of International Conference on Trends in Computational and Cognitive Engineering: Proceedings of TCCE 2020. Springer, pp. 341–354.
- Iso, I., 2011. Iec25010: 2011 systems and software engineering—systems and software quality requirements and evaluation (square)—system and software quality models. *Int. Organ. Stand.* 34 (2910), 208.
- Issa, B., Jasser, M.B., Chua, H.N., Hamzah, M., 2023. A comparative study on embedding models for keyword extraction using keybert method. In: 2023 IEEE 13th International Conference on System Engineering and Technology. ICSET, IEEE, pp. 40–45.
- Khan, A.A., Ahmad, A., Waseem, M., Liang, P., Fahmideh, M., Mikkonen, T., Abrahams-son, P., 2023. Software architecture for quantum computing systems—A systematic review. *J. Syst. Softw.* 201, 111682.
- Kocer, E., Ko, T.W., Behler, J., 2022. Neural network potentials: A concise overview of methods. *Annu. Rev. Phys. Chem.* 73 (1), 163–186.
- Luo, J., Zhao, J., 2025. Formalization of quantum intermediate representations for code safety. *J. Syst. Softw.* 219, 112236.
- Messina, P., 2017. The exascale computing project. *Comput. Sci. Eng.* 19 (3), 63–67.
- Murillo, J.M., Garcia-Alonso, J., Moguel, E., Barzen, J., Leymann, F., Ali, S., Yue, T., Arcaini, P., Castillo, R.P., de Guzmán, I.G.R., et al., 2024. Quantum software engineering: Roadmap and challenges ahead. *arXiv preprint arXiv:2404.06825*.
- Nielsen, M.A., Chuang, I.L., 2010. Quantum Computation and Quantum Information. Cambridge University Press.
- Nosouhi, M.R., Sood, K., Chamola, V., Jeong, J.J., Gaddam, A., 2023. Towards quantum-secure software defined networks. *IET Quantum Commun.*
- Park, K., Hong, J.S., Kim, W., 2020. A methodology combining cosine similarity with classifier for text classification. *Appl. Artif. Intell.* 34 (5), 396–411.
- Piattini, M., Peterssen, G., Pérez-Castillo, R., Hevia, J.L., Serrano, M.A., Hernández, G., De Guzmán, I.G.R., Paradelo, C.A., Polo, M., Murina, E., et al., 2020. The talavera manifesto for quantum software engineering and programming. In: QANSWER. pp. 1–5.
- Piattini, M., Serrano, M., Perez-Castillo, R., Petersen, G., Hevia, J.L., 2021. Toward a quantum software engineering. *IT Prof.* 23 (1), 62–66.
- Ravi, P., Chattopadhyay, A., Bhasin, S., 2022. Security and quantum computing: An overview. In: 2022 IEEE 23rd Latin American Test Symposium. LATS, IEEE, pp. 1–6.
- Röder, M., Both, A., Hinneburg, A., 2015. Exploring the space of topic coherence measures. In: Proceedings of the Eighth ACM International Conference on Web Search and Data Mining. pp. 399–408.
- Saki, A.A., Alam, M., Phalak, K., Suresh, A., Topaloglu, R.O., Ghosh, S., 2021. A survey and tutorial on security and resilience of quantum computing. In: 2021 IEEE European Test Symposium. ETS, IEEE, pp. 1–10.
- Santos, J.C., Peruma, A., Mirakhorli, M., Galstery, M., Vidal, J.V., Sejfia, A., 2017. Understanding software vulnerabilities related to architectural security tactics: An empirical investigation of chromium, php and thunderbird. In: 2017 IEEE International Conference on Software Architecture. ICSA, IEEE, pp. 69–78.
- Sun, S., 2011. Meta-analysis of Cohen's kappa. *Heal. Serv. Outcomes Res. Methodol.* 11, 145–163.
- Weder, B., Barzen, J., Leymann, F., Vietz, D., 2022. Quantum software development lifecycle. In: Quantum Software Engineering. Springer, pp. 61–83.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering. Springer Science & Business Media.
- Zhao, J., 2020. Quantum software engineering: Landscapes and horizons. *arXiv preprint arXiv:2007.07047*.